```
from time import *
from qiskit import QuantumCircuit
from qiskit.visualization import plot_bloch_multivector
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram, plot_gate_map,plot_error_map
```

Task 1

```
qc = QuantumCircuit(1)
qc.x(0)
qc.barrier(0)
qc.h(0)
qc.barrier(0)
qc.z(0)
qc.barrier(0)
qc.draw()
```



```
qc.save_statevector()
sim = AerSimulator()
result = sim.run(qc).result()
sv = result.get_statevector()
print(sv)
```

```
Statevector([0.+0.j, 0.-1.j])
```

```
plot_bloch_multivector(sv)
```



```
qc = QuantumCircuit(2,2)
qc.x(0)
qc.barrier(0,1)
qc.h(0)
qc.barrier(0,1)
qc.z(0)
qc.barrier(0,1)
qc.cx(0,1)
qc.barrier(0,1)
qc.measure(0,0)
qc.measure(1,1)
qc.draw()
```



```
qc.save_statevector()
sim = AerSimulator()
result = sim.run(qc).result()
sv = result.get_statevector()
print(sv)
```

```
Statevector([0.+0.j, 0.-1.j, 0.+0.j, 0.+0.j],
            dim=(2, 2))
```

```
plot_bloch_multivector(sv)
```



```
qc = QuantumCircuit(3,3)
qc.x(0)
qc.barrier(0,1,2)
qc.h(0)
qc.barrier(0,1,2)
qc.z(0)
qc.barrier(0,1,2)
qc.cx(0,1)
qc.barrier(0,1,2)
qc.ccx(0,1,2)
qc.barrier(0,1,2)
qc.measure(0,0)
qc.measure(1,1)
qc.measure(2,2)
qc.draw()
```



```
qc.save_statevector()
sim = AerSimulator()
result = sim.run(qc).result()
sv = result.get_statevector()
print(sv)
```

```
Statevector([0.+0.j, 0.-1.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j,
            0.+0.j],
            dim=(2, 2, 2))
```

```
plot_bloch_multivector(sv)
```



```
qc = QuantumCircuit(4,4)
qc.x(0)
qc.barrier(0,1,2,3)
qc.h(0)
qc.barrier(0,1,2,3)
qc.z(0)
qc.barrier(0,1,2,3)
qc.cx(0,1)
qc.barrier(0,1,2,3)
qc.ccx(0,1,2)
qc.barrier(0,1,2,3)
qc.cx(0,3)
qc.barrier(0,1,2,3)
qc.measure(0,0)
qc.measure(1,1)
qc.measure(2,2)
qc.measure(3,3)
qc.draw()
```



```
qc.save_statevector()
sim = AerSimulator()
result = sim.run(qc).result()
sv = result.get_statevector()
print(sv)
```

```
Statevector([0.+0.j, 0.-1.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j,
            0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j,
            0.+0.j, 0.+0.j],
            dim=(2, 2, 2, 2))
```

```
plot_bloch_multivector(sv)
```



```
qc = QuantumCircuit(5,5)
qc.x(0)
qc.barrier(0,1,2,3,4)
qc.h(0)
qc.barrier(0,1,2,3,4)
qc.z(0)
qc.barrier(0,1,2,3,4)
qc.cx(0,1)
qc.barrier(0,1,2,3,4)
qc.ccx(0,1,2)
qc.barrier(0,1,2,3,4)
qc.cx(0,3)
qc.barrier(0,1,2,3,4)
qc.cx(0,4)
qc.barrier(0,1,2,3,4)
qc.measure(0,0)
qc.measure(1,1)
qc.measure(2,2)
qc.measure(3,3)
qc.measure(4,4)
qc.draw()
```



```
qc.save_statevector()
sim = AerSimulator()
result = sim.run(qc).result()
sv = result.get_statevector()
print(sv)
```

```
Statevector([0.+0.j, 0.-1.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j,
            0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j,
            0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j,
            0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j,
            0.+0.j, 0.+0.j],
            dim=(2, 2, 2, 2, 2))
```

```
plot_bloch_multivector(sv)
```



```
qc.save_statevector()
sim = AerSimulator()
result = sim.run(qc).result()
sv = result.get_statevector()
print(sv)
```

```
Statevector([0.+0.j, 0.-1.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j,
            0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j,
            0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j,
            0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j,
            0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j,
            0.+0.j],
            dim=(2, 2, 2, 2, 2, 2))
```

```
plot_bloch_multivector(sv)
```



```
qc = QuantumCircuit(6,6)
qc.x(0)
qc.barrier(0,1,2,3,4,5)
qc.h(0)
qc.barrier(0,1,2,3,4,5)
qc.z(0)
qc.barrier(0,1,2,3,4,5)
qc.cx(0,1)
qc.barrier(0,1,2,3,4,5)
qc.ccx(0,1,2)
qc.barrier(0,1,2,3,4,5)
qc.cx(0,3)
qc.barrier(0,1,2,3,4,5)
qc.cx(0,4)
qc.barrier(0,1,2,3,4,5)
qc.cx(0,5)
qc.barrier(0,1,2,3,4,5)
qc.measure(0,0)
qc.measure(1,1)
qc.measure(2,2)
qc.measure(3,3)
qc.measure(4,4)
qc.measure(5,5)
qc.draw()
```



```
qc.save_statevector()
sim = AerSimulator()
result = sim.run(qc).result()
sv = result.get_statevector()
print(sv)
```

```
Statevector([0.+0.j, 0.-1.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j,
            0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j,
            0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j,
            0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j,
            0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j,
            0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j,
            0.+0.j, 0.+0.j],
            dim=(2, 2, 2, 2, 2, 2)) 
```

```
plot_bloch_multivector(sv)
```



```
qc.save_statevector()
sim = AerSimulator()
result = sim.run(qc).result()
sv = result.get_statevector()
print(sv)
```

```
Statevector([0.+0.j, 0.-1.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j,
            0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j,
            0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j,
            0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j,
            0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j,
            0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j,
            0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j,
            0.+0.j],
            dim=(2, 2, 2, 2, 2, 2, 2)) 
```

```
plot_bloch_multivector(sv)
```


TITOR Tasks for Laboratory 5

```
from qiskit import QuantumCircuit
from qiskit.visualization import plot_bloch_multivector
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram, plot_gate_map,plot_error_map
from qiskit_aer.noise
```

```
def qc_x(n,m):
    qc = QuantumCircuit(n,n)
    for i in range(m):
        qc.x(i)
        qc.barrier(range(n))
    return qc
qc = qc_x(3,2)
qc.draw()
```



```
def qc_h(n,m):
    qc = QuantumCircuit(n,n)
    for i in range(m):
        qc.h(i)
        qc.barrier(range(n))
    return qc
qc = qc_h(3,2)
qc.draw()
```



```
def qc_z(n,m):
    qc = QuantumCircuit(n,n)
    for i in range(m):
        qc.z(i)
        qc.barrier(range(n))
    return qc
qc = qc_z(3,2)
qc.draw()
```



```
def qc_cx(n,m):
    qc = QuantumCircuit(n,n)
    for i in range(m):
        qc.cx(i,i+1)
        qc.barrier(range(n))
    return qc
qc = qc_cx(3,2)
qc.draw()
```



```
def qc_ccx(n,m):
    qc = QuantumCircuit(n,n)
    for i in range(m):
        qc.ccx(i,i+1,i+2)
        qc.barrier(range(n))
    return qc
qc = qc_ccx(3,2)
qc.draw()
```



```
def qc_cz(n,m):
    qc = QuantumCircuit(n,n)
    for i in range(m):
        qc.cz(i,i+1)
        qc.barrier(range(n))
    return qc
qc = qc_cz(3,2)
qc.draw()
```



```
def qc_cccx(n,m):
    qc = QuantumCircuit(n,n)
    for i in range(m):
        qc.cccx(i,i+1,i+2,i+3)
        qc.barrier(range(n))
    return qc
qc = qc_cccx(4,2)
qc.draw()
```



```
def qc_cccz(n,m):
    qc = QuantumCircuit(n,n)
    for i in range(m):
        qc.cccz(i,i+1,i+2,i+3)
        qc.barrier(range(n))
    return qc
qc = qc_cccz(4,2)
qc.draw()
```



```
def qc_ccccx(n,m):
    qc = QuantumCircuit(n,n)
    for i in range(m):
        qc.ccccx(i,i+1,i+2,i+3,i+4)
        qc.barrier(range(n))
    return qc
qc = qc_ccccx(5,2)
qc.draw()
```



```
def qc_ccccz(n,m):
    qc = QuantumCircuit(n,n)
    for i in range(m):
        qc.ccccz(i,i+1,i+2,i+3,i+4)
        qc.barrier(range(n))
    return qc
qc = qc_ccccz(5,2)
qc.draw()
```



```
def qc_cccccx(n,m):
    qc = QuantumCircuit(n,n)
    for i in range(m):
        qc.cccccx(i,i+1,i+2,i+3,i+4,i+5)
        qc.barrier(range(n))
    return qc
qc = qc_cccccx(6,2)
qc.draw()
```



```
def qc_cccccz(n,m):
    qc = QuantumCircuit(n,n)
    for i in range(m):
        qc.cccccz(i,i+1,i+2,i+3,i+4,i+5)
        qc.barrier(range(n))
    return qc
qc = qc_cccccz(6,2)
qc.draw()
```



```
def qc_ccccccx(n,m):
    qc = QuantumCircuit(n,n)
    for i in range(m):
        qc.ccccccx(i,i+1,i+2,i+3,i+4,i+5,i+6)
        qc.barrier(range(n))
    return qc
qc = qc_ccccccx(7,2)
qc.draw()
```



```
def qc_ccccccz(n,m):
    qc = QuantumCircuit(n,n)
    for i in range(m):
        qc.ccccccz(i,i+1,i+2,i+3,i+4,i+5,i+6)
        qc.barrier(range(n))
    return qc
qc = qc_ccccccz(7,2)
qc.draw()
```



```
def qc_cccccccx(n,m):
    qc = QuantumCircuit(n,n)
    for i in range(m):
        qc.cccccccx(i,i+1,i+2,i+3,i+4,i+5,i+6,i+7)
        qc.barrier(range(n))
    return qc
qc = qc_cccccccx(8,2)
qc.draw()
```



```
def qc_cccccccz(n,m):
    qc = QuantumCircuit(n,n)
    for i in range(m):
        qc.cccccccz(i,i+1,i+2,i+3,i+4,i+5,i+6,i+7)
        qc.barrier(range(n))
    return qc
qc = qc_cccccccz(8,2)
qc.draw()
```



```
def qc_ccccccccx(n,m):
    qc = QuantumCircuit(n,n)
    for i in range(m):
        qc.ccccccccx(i,i+1,i+2,i+3,i+4,i+5,i+6,i+7,i+8)
        qc.barrier(range(n))
    return qc
qc = qc_ccccccccx(9,2)
qc.draw()
```



```
def qc_ccccccccz(n,m):
    qc = QuantumCircuit(n,n)
    for i in range(m):
        qc.ccccccccz(i,i+1,i+2,i+3,i+4,i+5,i+6,i+7,i+8)
        qc.barrier(range(n))
    return qc
qc = qc_ccccccccz(9,2)
qc.draw()
```



```
def qc_cccccccccx(n,m):
    qc = QuantumCircuit(n,n)
    for i in range(m):
        qc.cccccccccx(i,i+1,i+2,i+3,i+4,i+5,i+6,i+7,i+8,i+9)
        qc.barrier(range(n))
    return qc
qc = qc_cccccccccx(10,2)
qc.draw()
```



```
def qc_cccccccccz(n,m):
    qc = QuantumCircuit(n,n)
    for i in range(m):
        qc.cccccccccz(i,i+1,i+2,i+3,i+4,i+5,i+6,i+7,i+8,i+9)
        qc.barrier(range(n))
    return qc
qc = qc_cccccccccz(10,2)
qc.draw()
```



```
def qc_ccccccccccx(n,m):
    qc = QuantumCircuit(n,n)
    for i in range(m):
        qc.ccccccccccx(i,i+1,i+2,i+3,i+4,i+5,i+6,i+7,i+8,i+9,i+10)
        qc.barrier(range(n))
    return qc
qc = qc_ccccccccccx(11,2)
qc.draw()
```



```
def qc_ccccccccccz(n,m):
    qc = QuantumCircuit(n,n)
    for i in range(m):
        qc.ccccccccccz(i,i+1,i+2,i+3,i+4,i+5,i+6,i+7,i+8,i+9,i+10)
        qc.barrier(range(n))
    return qc
qc = qc_ccccccccccz(11,2)
qc.draw()
```



```
def qc_cccccccccccx(n,m):
    qc = QuantumCircuit(n,n)
    for i in range(m):
        qc.cccccccccccx(i,i+1,i+2,i+3,i+4,i+5,i+6,i+7,i+8,i+9,i+10,i+11)
        qc.barrier(range(n))
    return qc
qc = qc_cccccccccccx(12,2)
qc.draw()
```



```
def qc_cccccccccccz(n,m):
    qc = QuantumCircuit(n,n)
    for i in range(m):
        qc.cccccccccccz(i,i+1,i+2,i+3,i+4,i+5,i+6,i+7,i+8,i+9,i+10,i+11)
        qc.barrier(range(n))
    return qc
qc = qc_cccccccccccz(12,2)
qc.draw()
```



```
def qc_ccccccccccccx(n,m):
    qc = QuantumCircuit(n,n)
    for i in range(m):
        qc.ccccccccccccx(i,i+1,i+2,i+3,i+4,i+5,i+6,i+7,i+8,i+9,i+10,i+11,i+12)
        qc.barrier(range(n))
    return qc
qc = qc_ccccccccccccx(13,2)
qc.draw()
```

```
def qc_ccccccccccccz(n,m):
    qc = QuantumCircuit(n,n)
    for i in range(m):
        qc.ccccccccccccz(i,i+1,i+2,i+3,i+4,i+5,i+6,i+7,i+8,i+9,i+10,i+11,i+12)
        qc.barrier(range(n))
    return qc
qc = qc_ccccccccccccz(13,2)
qc.draw()
```



```
def qc_cccccccccccccx(n,m):
    qc = QuantumCircuit(n,n)
    for i in range(m):
        qc.cccccccccccccx(i,i+1,i+2,i+3,i+4,i+5,i+6,i+7,i+8,i+9,i+10,i+11,i+12,i+13)
        qc.barrier(range(n))
    return qc
qc = qc_cccccccccccccx(14,2)
qc.draw()
```



```
def qc_cccccccccccccz(n,m):
    qc = QuantumCircuit(n,n)
    for i in range(m):
        qc.cccccccccccccz(i,i+1,i+2,i+3,i+4,i+5,i+6,i+7,i+8,i+9,i+10,i+11,i+12,i+13)
        qc.barrier(range(n))
    return qc
qc = qc_cccccccccccccz(14,2)
qc.draw()
```



```
def qc_ccccccccccccccx(n,m):
    qc = QuantumCircuit(n,n)
    for i in range(m):
        qc.ccccccccccccccx(i,i+1,i+2,i+3,i+4,i+5,i+6,i+7,i+8,i+9,i+10,i+11,i+12,i+13,i+14)
        qc.barrier(range(n))
    return qc
qc = qc_ccccccccccccccx(15,2)
qc.draw()
```



```
def qc_ccccccccccccccz(n,m):
    qc = QuantumCircuit(n,n)
    for i in range(m):
        qc.ccccccccccccccz(i,i+1,i+2,i+3,i+4,i+5,i+6,i+7,i+8,i+9,i+10,i+11,i+12,i+13,i+14)
        qc.barrier(range(n))
    return qc
qc = qc_ccccccccccccccz(15,2)
qc.draw()
```



```
def qc_cccccccccccccccx(n,m):
    qc = QuantumCircuit(n,n)
    for i in range(m):
        qc.cccccccccccccccx(i,i+1,i+2,i+3,i+4,i+5,i+6,i+7,i+8,i+9,i+10,i+11,i+12,i+13,i+14,i+15)
        qc.barrier(range(n))
    return qc
qc = qc_cccccccccccccccx(16,2)
qc.draw()
```



```
def qc_cccccccccccccccz(n,m):
    qc = QuantumCircuit(n,n)
    for i in range(m):
        qc.cccccccccccccccz(i,i+1,i+2,i+3,i+4,i+5,i+6,i+7,i+8,i+9,i+10,i+11,i+12,i+13,i+14,i+15)
        qc.barrier(range(n))
    return qc
qc = qc_cccccccccccccccz(16,2)
qc.draw()
```



```
def qc_ccccccccccccccccx(n,m):
    qc = QuantumCircuit(n,n)
    for i in range(m):
        qc.ccccccccccccccccx(i,i+1,i+2,i+3,i+4,i+5,i+6,i+7,i+8,i+9,i+10,i+11,i+12,i+13,i+14,i+15,i+16)
        qc.barrier(range(n))
    return qc
qc = qc_ccccccccccccccccx(17,2)
qc.draw()
```



```
def qc_ccccccccccccccccz(n,m):
    qc = QuantumCircuit(n,n)
    for i in range(m):
        qc.ccccccccccccccccz(i,i+1,i+2,i+3,i+4,i+5,i+6,i+7,i+8,i+9,i+10,i+11,i+12,i+13,i+14,i+15,i+16)
        qc.barrier(range(n))
    return qc
qc = qc_ccccccccccccccccz(17,2)
qc.draw()
```



```
def qc_cccccccccccccccccx(n,m):
    qc = QuantumCircuit(n,n)
    for i in range(m):
        qc.cccccccccccccccccx(i,i+1,i+2,i+3,i+4,i+5,i+6,i+7,i+8,i+9,i+10,i+11,i+12,i+13,i+14,i+15,i+16,i+17)
        qc.barrier(range(n))
    return qc
qc = qc_cccccccccccccccccx(18,2)
qc.draw()
```



```
def qc_cccccccccccccccccz(n,m):
    qc = QuantumCircuit(n,n)
    for i in range(m):
        qc.cccccccccccccccccz(i,i+1,i+2,i+3,i+4,i+5,i+6,i+7,i+8,i+9,i+10,i+11,i+12,i+13,i+14,i+15,i+16,i+17)
        qc.barrier(range(n))
    return qc
qc = qc_cccccccccccccccccz(18,2)
qc.draw()
```